

CAR RENTAL & RESERVATION SYSTEM

Pre-Development Technical Documentation

Object Oriented Programming (CS-102-L) | Spring 2026 | Section B
Instructor: Sir Faheem Saleem

Team Members

Wajeeh ul Hassan — 25-CS-02
Hafiz M. Saeed — 25-CS-05
M. Irfan Yasir — 25-CS-194
Hamza Khurram — 25-CS-128

Table of Contents

Table of Contents.....	2
1. Project Overview.....	4
1.1 What We Are Building.....	4
1.2 Instructor Feature Requirements	4
2. Architectural Approach — Options & Decision	4
2.1 Option A — Monolithic (Single File)	4
2.2 Option B — Header-Only (.h files)	5
2.3 Option C — Separated .h + .cpp Per Class (Chosen)	5
3. File Structure.....	6
3.1 Full Directory Layout.....	6
3.2 The Golden Rule of main.cpp	7
3.3 Header File (.h) Structure	7
4. Class Hierarchy & Relationships	8
4.1 Vehicle Hierarchy	8
4.2 User Hierarchy.....	9
4.3 Transaction Hierarchy.....	9
4.4 Standalone Classes.....	9
5. OOP Principles — Where, When, and How to Apply Each.....	9
5.1 Encapsulation.....	10
5.2 Inheritance.....	10
5.3 Polymorphism.....	11
5.4 Abstraction	11
5.5 Composition	12
5.6 Function Overloading.....	12
5.7 File Handling	13
6. Memory Management	14
7. Key Design Decisions	14
7.1 Why FileHandler is a Separate Class	14
7.2 Why vector<Vehicle*> Instead of an Array.....	15
7.3 Why Pipe () as File Delimiter	15
7.4 Why Transaction is a Separate Hierarchy.....	15
7.5 Why InspectionReport Uses Composition Not Inheritance.....	15
8. Suggested Team Division of Work	15
9. Pre-Coding Checklist	16
10. Quick Reference — OOP Concept Locator	17

1. Project Overview

This document is the complete pre-development reference for the Car Rental and Reservation System (ACRRS). It is intended to be read by all four team members before a single line of code is written. It covers every architectural decision, every design choice, and why each choice was made — so the team can code with confidence rather than guessing.

1.1 What We Are Building

A console-based C++ application that manages a fleet of 10+ vehicles, supports vehicle rental and sale transactions, provides a trip planner and search engine, records inspection reports, and persists all data between program runs using text files. The system has two user roles: Admin and Customer, each with their own menu and capabilities.

1.2 Instructor Feature Requirements

The following six features were specifically requested by the lab instructor and must be implemented:

- Sale / Purchase module alongside the existing rental system
- 10+ vehicles loaded and manageable in the fleet
- Inspection Sheet generated on every vehicle return
- Evaluation Remarks attached to rentals and inspections
- Trip Planner that recommends vehicles based on distance
- Vehicle Search with multiple filter options

2. Architectural Approach — Options & Decision

Before writing any code, the team must agree on one structural approach. Three approaches were evaluated.

2.1 Option A — Monolithic (Single File)

All code lives in one main.cpp file. No separate headers or implementation files.

Aspect	Detail
Compilation	One command: g++ main.cpp -o app

Aspect	Detail
Teamwork	Only one person can work at a time — merge conflicts on every edit
Scalability	Unusable past ~400 lines; impossible to navigate
OOP visibility	Instructor cannot see clear class separation
Verdict	Rejected — unsuitable for team projects of this size

2.2 Option B — Header-Only (.h files)

Each class is written entirely inside its .h header file — both the declaration and the full method implementation in the same file.

Aspect	Detail
Compilation	Include headers in main.cpp; still fairly simple
Teamwork	Better — each person owns a .h file
OOP visibility	Moderate — all in one file per class
Technical issue	Definitions in headers get compiled multiple times, causing linker errors in large projects
Industry standard	Not used professionally; bad habit to teach students
Verdict	Possible for small projects, but rejected in favour of Option C

2.3 Option C — Separated .h + .cpp Per Class (Chosen)

Each class gets exactly two files: a header (.h) for the class declaration, and an implementation (.cpp) for the method definitions. This is the professional C++ standard.

Aspect	Detail
Compilation	g++ src/*.cpp -I include -o app (one command)
Teamwork	Each teammate owns a class pair independently
OOP visibility	Maximum — instructor immediately sees clean separation
Technical correctness	Declarations in .h, definitions in .cpp — no linker issues
Industry standard	Exactly how professional C++ projects are structured
Verdict	Chosen — this is our approach

Why Option C Wins for a 4-Person Team

Person 1 writes Vehicle.h + Vehicle.cpp. Person 2 writes User.h + User.cpp.

They work simultaneously with zero conflicts because they touch different files.

The instructor opens the include/ folder and immediately sees the full class architecture.

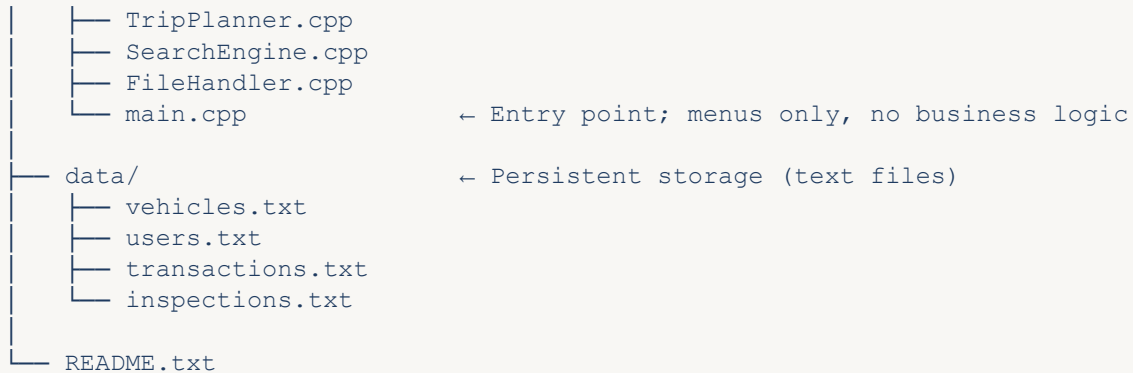
Each .h file is the 'contract' — any teammate can read it to understand a class without reading the implementation. This is real-world software engineering practice.

3. File Structure

The project is organised into three top-level folders. Every file has a specific role and a specific owner on the team.

3.1 Full Directory Layout

```
CarRentalSystem/
├── include/                                ← All .h header files (class declarations)
│   ├── Vehicle.h
│   ├── Economy.h
│   ├── Luxury.h
│   ├── SUV.h
│   ├── User.h
│   ├── Admin.h
│   ├── Customer.h
│   ├── Transaction.h
│   ├── RentalTransaction.h
│   ├── SaleTransaction.h
│   ├── InspectionReport.h
│   ├── TripPlanner.h
│   ├── SearchEngine.h
│   └── FileHandler.h
├── src/                                    ← All .cpp implementation files
│   ├── Vehicle.cpp
│   ├── Economy.cpp
│   ├── Luxury.cpp
│   ├── SUV.cpp
│   ├── User.cpp
│   ├── Admin.cpp
│   ├── Customer.cpp
│   ├── Transaction.cpp
│   ├── RentalTransaction.cpp
│   ├── SaleTransaction.cpp
│   └── InspectionReport.cpp
```



3.2 The Golden Rule of main.cpp

main.cpp must contain only: program startup, loading data, the top-level menu loop, and shutdown/save. If you find yourself writing business logic in main.cpp, it belongs in a class instead. A correctly written main.cpp will be under 80 lines total.

```
// Correct main.cpp structure
int main() {
    FileHandler fh;
    vector<Vehicle*> fleet = fh.loadVehicles();
    vector<User*> users = fh.loadUsers();

    int choice;
    do {
        showMainMenu();
        cin >> choice;
        handleChoice(choice, fleet, users);
    } while (choice != 0);

    fh.saveVehicles(fleet);
    fh.saveUsers(users);
    return 0;
}
```

3.3 Header File (.h) Structure

Every header file must follow the same pattern: include guard, includes, class declaration, then closing brace. Never put method implementations in headers.

```
#ifndef VEHICLE_H          // Include guard – prevents double inclusion
#define VEHICLE_H

#include <string>
```

```

#include <iostream>
using namespace std;

class Vehicle {
private:
    string vehicleID;    // Data members are ALWAYS private
    string model;
    float  rentalRate;
    bool   isAvailable;

public:
    Vehicle(string id, string model, float rate); // Constructor
    virtual ~Vehicle();                          // Virtual destructor

    // Getters
    string getID() const;
    string getModel() const;
    float  getRentalRate() const;
    bool   getAvailable() const;

    // Setters
    void setAvailable(bool status);

    // Pure virtual — makes Vehicle abstract
    virtual float calculateCost(int hours) = 0;
    virtual void  displayInfo()           = 0;
};

#endif // VEHICLE_H

```

4. Class Hierarchy & Relationships

The system has three inheritance trees and three standalone utility classes. Understanding the relationships between them is essential before coding begins.

4.1 Vehicle Hierarchy

Class	Type	Inherits From	What It Adds
Vehicle	Abstract base	—	vehicleID, model, rentalRate, isAvailable; pure virtual calculateCost(), displayInfo()
Economy	Concrete	Vehicle	No extras; low daily rate multiplier
Luxury	Concrete	Vehicle	luxuryFeatures (string list); higher rate multiplier
SUV	Concrete	Vehicle	seatingCapacity (int); mid rate multiplier

4.2 User Hierarchy

Class	Type	Inherits From	What It Adds
User	Abstract base	—	userID, name, phone; pure virtual showMenu()
Admin	Concrete	User	Can add/remove vehicles, view all records
Customer	Concrete	User	rentalHistory vector; can rent, return, plan trips

4.3 Transaction Hierarchy

Class	Type	Inherits From	What It Adds
Transaction	Abstract base	—	transactionID, vehicle ref, customer ref, date; pure virtual finalise()
RentalTransaction	Concrete	Transaction	startTime, endTime, hoursRented; calculates rental bill
SaleTransaction	Concrete	Transaction	salePrice; records one-time vehicle purchase

4.4 Standalone Classes

Class	Relationship	Purpose
InspectionReport	Composition — HAS a Vehicle reference	Generated when a vehicle is returned; stores fuel level, damage notes, mileage, evaluator remarks
TripPlanner	Uses Vehicle data (no inheritance)	Takes source, destination, estimated distance; recommends vehicle category and estimates cost
SearchEngine	Operates on vector<Vehicle*>	Provides overloaded search() methods: by name, category, price range, availability
FileHandler	Uses all other classes	Centralised file I/O; all save/load operations go through this class only

5. OOP Principles — Where, When, and How to Apply Each

This section maps each OOP concept to its exact location in the project. Do not apply these concepts randomly — each has a specific home and a specific purpose.

5.1 Encapsulation

Definition: Keep data private; expose it only through controlled public methods.

Where to apply: Every class in the project. This is not optional — it is the foundation.

Rule: All data members are private. Access goes through getters and setters. Sensitive fields (e.g., passwords, financial totals) have getters but no public setters.

```
// WRONG — public data member
class Vehicle {
public:
    float rentalRate;    // anyone can change this directly
};

// CORRECT — encapsulated
class Vehicle {
private:
    float rentalRate;
public:
    float getRentalRate() const { return rentalRate; }
    void setRentalRate(float r) { if (r > 0) rentalRate = r; } // validated
};
```

5.2 Inheritance

Definition: A child class receives all data members and methods of its parent class and can add its own.

Where to apply: The three class trees — Vehicle/Economy/Luxury/SUV, User/Admin/Customer, Transaction/RentalTransaction/SaleTransaction.

Decision rule: Only use inheritance when the child truly IS a type of the parent. Economy IS a Vehicle. RentalTransaction IS a Transaction. Do not inherit just to reuse code — that is composition's job.

```
// Economy.h
class Economy : public Vehicle {
public:
    Economy(string id, string model);
    float calculateCost(int hours) override; // Economy rate logic
    void displayInfo() override;
    // No extra data — Economy vehicles have no special attributes
};

// Luxury.h
class Luxury : public Vehicle {
private:
```

```

        string luxuryFeatures;           // Something unique to Luxury
    public:
        Luxury(string id, string model, string features);
        float calculateCost(int hours) override; // Luxury rate — higher
        multiplier
        void displayInfo() override;
        string getLuxuryFeatures() const;
};

```

5.3 Polymorphism

Definition: One interface, many implementations. A base class pointer calls the correct overridden method at runtime.

Where to apply: The `calculateCost()` and `displayInfo()` virtual functions in `Vehicle`. The `showMenu()` virtual function in `User`. The `finalise()` virtual function in `Transaction`.

The key pattern: Store all vehicles in a `vector<Vehicle*>`. Loop through them and call methods — C++ automatically invokes the right version for each object type.

```

// In main or a manager class:
vector<Vehicle*> fleet;
fleet.push_back(new Economy("E001", "Toyota Corolla"));
fleet.push_back(new Luxury("L001", "BMW 7 Series", "Heated seats, Sunroof"));
fleet.push_back(new SUV("S001", "Toyota Fortuner", 7));

// Polymorphic call — correct version runs for each type automatically
for (Vehicle* v : fleet) {
    v->displayInfo();           // calls Economy::displayInfo or
    Luxury::displayInfo etc.
    v->calculateCost(24);       // each type applies its own rate logic
}

```

5.4 Abstraction

Definition: Hide implementation complexity behind a simple interface. Enforced in C++ by making a class abstract (at least one pure virtual function).

Where to apply: `Vehicle`, `User`, and `Transaction` are all abstract. You cannot create a raw `Vehicle` object — this models reality correctly. There is no such thing as a generic vehicle without a type.

```

// Vehicle is abstract because of = 0
virtual float calculateCost(int hours) = 0; // Pure virtual
virtual void displayInfo()             = 0; // Pure virtual

```

```
// This will give a compile error – good!
Vehicle v;           // ERROR: cannot instantiate abstract class

// This is correct
Vehicle* v = new Economy("E001", "Corolla"); // OK – concrete type
```

5.5 Composition

Definition: A class is built from objects of other classes as member variables. Models 'HAS A' relationships.

Where to apply: InspectionReport HAS a Vehicle reference and a Customer reference.
RentalTransaction HAS a Vehicle and a Customer. TripPlanner uses Vehicle objects internally.

Why not inheritance here: An InspectionReport is NOT a Vehicle. It is not a User. It contains them. Using inheritance here would be wrong and would confuse the examiner.

```
// InspectionReport.h
class InspectionReport {
private:
    Vehicle*   inspectedVehicle; // HAS a vehicle – composition
    Customer*  inspector;        // HAS a customer – composition
    string     fuelLevel;
    string     damageNotes;
    float      mileage;
    string     evaluationRemarks;
    string     date;
public:
    InspectionReport(Vehicle* v, Customer* c);
    void fillReport();
    void displayReport() const;
    void saveToFile(ofstream& out) const;
};
```

5.6 Function Overloading

Definition: Multiple functions with the same name but different parameter lists. The compiler picks the right one at compile time.

Where to apply: SearchEngine class — multiple search() methods with different filter parameters.

```
// SearchEngine.h
class SearchEngine {
private:
```

```

        vector<Vehicle*>& fleet;
public:
    SearchEngine(vector<Vehicle*>& f);

    // Overloaded search() — same name, different parameters
    vector<Vehicle*> search(string name);           // by model name
    vector<Vehicle*> search(float maxPrice);        // by max price
    vector<Vehicle*> search(string category, bool available); // by type +
status
    vector<Vehicle*> search(float minPrice, float maxPrice); // by price
range
};

```

5.7 File Handling

Where to apply: Isolated entirely in the FileHandler class. No other class should directly open files. This makes debugging trivial — if file data is corrupted, only one file needs to be inspected.

```

// FileHandler.h
#include <fstream>
class FileHandler {
public:
    vector<Vehicle*> loadVehicles();
    void saveVehicles(const vector<Vehicle*>& fleet);

    vector<User*>    loadUsers();
    void saveUsers(const vector<User*>& users);

    vector<Transaction*> loadTransactions();
    void saveTransactions(const vector<Transaction*>& txns);

    void saveInspection(const InspectionReport& report);
};

```

Text file format — use pipe (|) as delimiter to avoid conflicts with spaces in names:

```

// vehicles.txt format
E|E001|Toyota Corolla|15.5|1
L|L001|BMW 7 Series|75.0|1
S|S001|Toyota Fortuner|45.0|0
// Format: Type|ID|Model|RatePerHour|IsAvailable

// On load: read first character to decide which derived class to construct
if (type == 'E') fleet.push_back(new Economy(id, model, rate));
else if (type == 'L') fleet.push_back(new Luxury(id, model, rate));
else if (type == 'S') fleet.push_back(new SUV(id, model, rate));

```

6. Memory Management

Because we use virtual functions, Vehicle must have a virtual destructor. Because we use new to create objects, we must delete them when done. Forgetting this causes memory leaks.

```
// Vehicle.h – virtual destructor is mandatory
virtual ~Vehicle() {}

// At program shutdown – delete all heap-allocated objects
for (Vehicle* v : fleet) {
    delete v;    // calls correct destructor for Economy/Luxury/SUV
}
fleet.clear();

for (User* u : users) {
    delete u;
}
users.clear();
```

Critical Rule

Every new must have a matching delete.

The virtual ~Vehicle() destructor ensures that when you call delete on a Vehicle* pointer, the correct derived class destructor runs — not just the base class one.

Without it, you get undefined behaviour and partial destruction.

7. Key Design Decisions

These are deliberate choices made before coding. Each one has a reason. Do not change them during development without discussing with the team.

7.1 Why FileHandler is a Separate Class

Option considered: Each class manages its own file I/O (Vehicle::saveToFile(), User::saveToFile(), etc.).

Problem: File logic scattered across 8 classes. If the file format changes, 8 files need editing. Bugs are hard to find.

Decision: One FileHandler class owns all I/O. One place to fix. One place to test. All other classes simply provide data through their getters.

7.2 Why `vector<Vehicle*>` Instead of an Array

Option considered: `Vehicle* fleet[50]` — fixed-size array.

Problem: Hard-coded size. Cannot grow. Must track a separate count variable manually.

Decision: `vector<Vehicle*> fleet` — grows automatically, has `.size()`, works with range-for loops, and pairs naturally with polymorphism (stores base class pointers to any derived type).

7.3 Why Pipe `()` as File Delimiter

Options considered: comma (,), tab (`\t`), semicolon (;), pipe `()`.

Problem with comma: Vehicle names and features can contain commas (e.g., 'Heated seats, Sunroof'). Comma delimiter breaks on these.

Decision: Pipe `()` — never appears in natural language names or descriptions. Simple to split with `getline(stream, field, '|')`.

7.4 Why Transaction is a Separate Hierarchy

Option considered: Add rental and sale methods directly to `Vehicle` or `Customer`.

Problem: A transaction involves both a `Vehicle` and a `Customer`. It belongs to neither — it is its own concept. Putting it in `Vehicle` mixes the 'what a vehicle is' with 'what happens during a deal'.

Decision: Transaction is its own class hierarchy with composition (contains `Vehicle` and `Customer` references). This is clean OOP modelling.

7.5 Why `InspectionReport` Uses Composition Not Inheritance

Decision test: Is an `InspectionReport` a type of `Vehicle`? No. Is it a type of `Customer`? No. Does it contain both? Yes.

Conclusion: Composition is correct. `InspectionReport` HAS a `Vehicle` reference and a `Customer` reference as member pointers.

8. Suggested Team Division of Work

With four members and separated .h/.cpp files, work can be parallelised almost entirely. The only dependency is that teammates must agree on class interfaces (the .h files) before writing implementations.

Member	Owns	Dependencies to agree on first
Member 1	Vehicle.h/.cpp, Economy, Luxury, SUV	The Vehicle interface — all others depend on this. Start here, Day 1.
Member 2	User.h/.cpp, Admin, Customer	User interface. Coordinate with Member 4 on how Customer links to Transaction.
Member 3	Transaction, RentalTransaction, SaleTransaction, InspectionReport	Needs Vehicle* and Customer* types — agree interfaces with M1 and M2 first.
Member 4	FileHandler, SearchEngine, TripPlanner, main.cpp, data files	Needs all class interfaces finalised. Works on data format agreement on Day 1.

Day 1 Must-Do (All Four Members Together)

1. Agree on the Vehicle interface — what methods and data it exposes publicly.
2. Agree on the data file format (delimiter, field order for each file).
3. Create all .h files with just declarations (no implementations yet).
4. Each member then writes their .cpp independently without blocking others.

9. Pre-Coding Checklist

Before writing any implementation code, the team should be able to answer yes to every item below.

1. Have all four members read this document in full?
2. Is the folder structure (include/, src/, data/) created on all machines?
3. Has the team agreed on the exact Vehicle public interface?
4. Has the team agreed on the pipe-delimited file format for all four data files?
5. Does every member know which .h and .cpp files they own?
6. Does every member understand the difference between inheritance and composition and can apply it correctly?
7. Is VS Code set up with the correct include path (g++ -I include)?
8. Has a test vehicles.txt been created with at least 10 sample entries?

9. Has the team agreed that main.cpp will contain only menus — no business logic?
10. Does every member know that all data members must be private?

10. Quick Reference — OOP Concept Locator

Use this table during development to find where each concept lives in the codebase.

OOP Concept	Where in This Project	Key Files
Encapsulation	All classes — private data + public methods	Every .h file
Inheritance	Vehicle tree, User tree, Transaction tree	Economy/Luxury/SUV.h, Admin/Customer.h, RentalTransaction/SaleTransaction.h
Polymorphism	vector<Vehicle*> loop calling calculateCost() and displayInfo()	main.cpp, Vehicle.h and derived .h files
Abstraction	Vehicle, User, Transaction (pure virtual = 0)	Vehicle.h, User.h, Transaction.h
Composition	InspectionReport contains Vehicle* and Customer*	InspectionReport.h
Function Overloading	SearchEngine::search() with different parameter signatures	SearchEngine.h
Virtual Destructor	~Vehicle(), ~User(), ~Transaction() are all virtual	Vehicle.h, User.h, Transaction.h
File Handling	FileHandler class — sole owner of all I/O	FileHandler.h, FileHandler.cpp
Dynamic Allocation	new Economy/Luxury/SUV, stored in vector<Vehicle*>	FileHandler.cpp (on load), main.cpp
Memory Cleanup	delete loop over all vectors before program exits	main.cpp